

Visibility-Constrained Faction Control Problem

Wachirawich Kanil
Chulalongkorn University
Student ID 6531339621
Github Username: ZigmaZero
Computational Geometry 2025
wachirawichkanil@gmail.com

Abstract

Visibility is a fundamental construct in computational geometry. In this study, we defined the visibility-constrained faction control problem and decided on a $O(P^3S^2)$ time baseline algorithm to find the solution. We then made improvements to this approach and recorded a 10-times improvement in runtime with significant efficiency in handling geometry complexity. We finally deliberate on applications of the process for real-world usage.

Keywords: Visibility polygon, Polygon arithmetic, Optimization

1. Problem Description

1.1. Introduction

Visibility is a fundamental construct in computational geometry, describing the observable subregion from a given point under line-of-sight constraints. This concept has been widely studied due to its relevance in robotics, surveillance, and computer graphics.

One prominent application arises in “fog-of-war” systems in strategy and simulation games, where information about the playing domain is revealed only through localized perception. In such games, multiple agents, grouped into factions, simultaneously observe the environment from distinct positions, each of them having different constraints on visibility.

While determining visible regions is already nontrivial in the presence of complex obstacles, a more intricate problem emerges when the visible domain is to be partitioned into regions controlled by individual agents or factions. This leads to a geometric formulation in which each point in the domain is assigned to the closest agent that can see it. Unlike classical Voronoi diagrams, where all sites expand outward globally under a uniform distance metric, this model introduces a restriction on competition, limiting it to only sites with direct line of sight. The resultant partition depends jointly on the distance metric and the combinatorial structure of visibility, producing regions that may be highly non-convex, disconnected, and sensitive to obstacle configuration.

The objective of this work is to formalize and analyze this visibility-constrained faction control problem. Given a finite planar region with polygonal obstacles and a set of agents partitioned into factions, we seek to compute the subregions controlled by each agent and, by extension, each faction. This problem raises both theoretical and algorithmic challenges due to the interaction of the distance between agents and visibility constraints.

1.2. Formal Definition

We define first the following terms to model the game upon which we demonstrate the algorithm.

Definition 1 (Playing Region). The playing region Ω is the finite 2D region $\{(o_x, o_y) \mid 0 \leq o_x \leq \Omega_x, 0 \leq o_y \leq \Omega_y\}$.

Definition 2 (Agent). An agent p is a point $(p_x, p_y) \in \Omega$.

Definition 3 (Obstacle). An obstacle O is a simple polygon where all vertices are points in Ω .

Definition 4 (Visibility). A point r in Ω is considered *visible* relative to an agent p if the line segment rp does not intersect any obstacles. We also say that p sees r . The *visible region* of p is the set of all point p sees. Formally, $\Lambda_p := \{r \in \Omega \mid \forall O \in \Sigma_O / (pr) \cap O = \emptyset\}$.

Definition 5 (Faction). A faction F is a finite set of agents. For every agent p , there exists exactly one faction F that contains p . We say that p belongs to F .

Definition 6 (Control). An agent p controls the point r if r is in Λ_p and for every other agent q that sees r (that is, r is in Λ_q), the distance $d(p, r) \leq d(q, r)$. The *control region* Δ_p of p is the set of all points p controls. Formally, $\Delta_p := \{r \in \Lambda_p \mid \forall q \in \Sigma_P (p = q \vee r \notin \Lambda_q \vee d(p, r) \leq d(q, r))\}$.

A faction $F = \{a, b, c, \dots\}$ controls the subregion $\Delta_F = \Delta_a \cup \Delta_b \cup \Delta_c \cup \dots$

We now define the program's specifications according to the problem model as shown in Table 1.

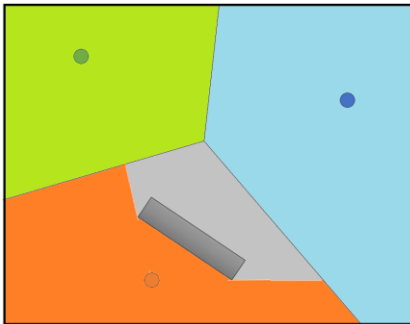
Table 1. Program Specifications

Detail	Description
Input	Size of the playing region Ω_x, Ω_y The number of obstacles M M obstacles, each of which is a list of points in Ω The number of factions F The number of agents P P agents, including their position and the faction they belong to.
Output	P subregions that each agent controls. F subregions that each faction controls.
Constraints	$1 \leq \Omega_x, \Omega_y \leq 1,000$ $0 \leq M \leq 100$ No obstacles overlap For each obstacle $O, 3 \leq V(O) \leq 10$ $1 \leq F \leq 4$ $F \leq P \leq 40$ Epsilon = 0.001 No two agents have distance < Epsilon

1.3. Challenges

We may be tempted to globally compute the Voronoi diagram for agents on the field for this problem. The rationale is that the control region appears to be the intersection of the agent's Voronoi cell and their visible region. However, Figure 1 demonstrates why this method is incorrect.

Figure 1. Incorrect Derivation of Control Regions



As can be seen in the image, the neutral area (in gray) is visible by the green and blue agents, which means their control regions must overlap it in some form. However, as the neutral region is within the Voronoi cell of the non-participating orange agent, this method fails to include said region into the other two's control regions.

Even if the construction algorithm is highly optimized, intersection is performed with $O(nm)$ in the worst case with respect to the vertex count of the initial polygons. While we could delegate the union to the renderer, considering the intersection could be incredibly complex, this method is far too slow to be usable in real-time. We must also aggressively cull unnecessary calculations and incrementally update each region with considerations to real-time changes and rendering.

There are no particular restrictions on obstacle positions relative to each other, so it is possible that obstacles may partially or fully intersect with each other. Further, tiny gaps in obstacles may result in inaccuracies in the computed regions due to the numerical precision errors. It is important that the implementation recognizes and adapts to this degenerate case and outputs the correct value.

2. Baseline Process

From Definition 6, we can adjust the form of the control region as follows; by letting $D_{pq} = \{r \in \Omega \mid d(p,r) > d(q,r)\}$, we find $\Delta_p = \{r \in \Lambda_p \mid \forall q \in \Sigma_P(p = q \vee r \notin \Lambda_q \vee d(p,r) \leq d(q,r))\}$
 $= \{r \in \Lambda_p \mid \forall q \in \Sigma_P(p = q \vee r \notin \Lambda_q \vee r \notin D_{pq})\}$
 $= \Lambda_p - \{r \in \Lambda_p \mid \forall q \in \Sigma_P(p \neq q \wedge r \in \Lambda_q \cap D_{pq})\}$
 $= \Lambda_p - \bigcup_{p \neq q} (\Lambda_q \cap D_{pq})$

We can now straightforwardly calculate the control region: we first calculate Λ for every agent on the field. Then, for every agent p , we compute $\Lambda_q \cap D_{pq}$ for every other agent q . We can then union all of the computed regions and subtract that from Λ_p to obtain Δ_p .

Of course, we must still deliberate on our choice of algorithm usage to generate each subregion in turn. For the baseline approach, we can obtain Λ_p by casting a ray to all obstacle and region edge endpoints and obtaining the minimum distance edge intersection, then sorting these intersections to form the polygon. D_{pq} is obtained by splitting Ω by pq 's perpendicular bisector, and the rest of the process is simple multi-polygon arithmetic.

Obtaining the visibility polygon in this way is $O(n^2)$ respective to the number of edges. Constructing D_{pq} is $O(1)$ per agent pair. The intersection of polygons are $O(n+m)$ respective to the vertices of each polygon, so the computation of $\Lambda_q \cap D_{pq}$ is essentially $O(n)$ respective to the vertices of Λ_q . The result of this calculation does not overlap often between each viable agent, so it can be assumed that $\bigcup_{p \neq q} (\Lambda_q \cap D_{pq})$ can be calculated in $O(n \log n)$ respective to the total vertices of all involved polygons. This result is then subtracted from Λ_p which will also take $O(n \log n)$ respective to the total vertices of the union and Λ_p . All the processes to derive Δ_p must then be repeated for each agent.

To approximate the time complexity of this process is an ordeal. First, we must approximate the vertex count of all intermediate polygons involved in the process. Letting $S = |E(\Sigma O)|$ be the total edges of all obstacles, then we may assume $|E(\Lambda_p)| = c_p S$ for some coefficient c which signifies the ratio of edge endpoints visible to p . The edge count of $\Lambda_q \cap D_{pq}$ can also be treated as some ratio of $c_q S$. The total vertices of all polygons involved in the union would be $P c_q S$, P being the number of agents. We can let $k P c_p S$ be the union result's number of edges, where $k = |E(\bigcup_{p \neq q} (\Lambda_q \cap D_{pq}))| / \sum_{p \neq q} |E(\Lambda_q \cap D_{pq})|$.

We now approximate the time complexity of the baseline process using these estimates. Obtaining Λ_p takes $O(S^2)$ time. Obtaining $\Lambda_q \cap D_{pq}$ takes $O(c_p S)$ time per each of P agents, and computing the union takes $O(P c_p S)$ time. The subsequent subtraction takes $O(c_p S(1+kP) \log(c_p S(1+kP)))$ time. We repeat the process to obtain Δ_p per each of P agents, so the entire process to obtain all agents' control regions has a time complexity of $O(S^2 P + S P^2 c_p + P c_p S(1+kP) \log(P c_p S(1+kP)))$.

To obtain all factions' control regions, we must compute the union of each agent under each faction's control regions. This is $O(P c_p S(1+kP) \log(c_F P c_p S(1+kP)))$ where $c_F = 1/F$ is the average ratio of players in each faction. The baseline process, as a sum of these two procedures, runs with time complexity roughly equal to the above expression.

Therefore, the baseline time complexity is quadratic with respect to both the number of agents and the complexity of polygons. In the worst case where the final subtraction is particularly complicated due to the presence of multiple separate polygons within the union having many edge intersections with the visibility

polygon, the time complexity may be cubic with respect to the number of agents.

The process can be written in pseudocode as follows:

```

R := Polygon[0, 0, Rx, Ry] // Region Bounds
F := List of Factions
P := List of Agents
O := List of Obstacles

Func VisibilityPolygon (Agent p)
-> Polygon:
Edges = Edges of polygon o for o in (O, R)
Vertices = Endpoints of Edges
Angles = Orientation of Vertices from p
Rays = Lines from p towards an extent at Angles
Sort (Angles, Rays) by Angles
Intersections = List()
For Ray in Rays:
Minpoint = null
For Edge in Edges:
If Edge intersects Ray at Point r:
If dist(p, r) < dist(p, Minpoint)
Minpoint = r

If Minpoint is not null:
Append Minpoint to Intersections
Return Polygon (Intersections)

Func DistanceHalfPlane(Agent p, Agent q)
-> Polygon:
M(mx,my) = midpoint of line pq
D(dx,dy) = direction vector of pq
P(px,py) = perpendicular vector from D
ND(ndx,ndy) = normalized D
NP(npx,npy) = normalized P
RD(rdx,rdy) = ND scaled to a large extent
RP(rpx,rpy) = NP scaled to a large extent
Return Polygon([M+RP, M-RP, M-RP+RD, M+RP+RD])

VPs = List()
For agent in P:
Append VisibilityPolygon(agent) to VPs

AgentControls = List()
For p in P:
Clips = List()
For q in P:
If q = p:
Continue
VQ = VPs[index of q in P]
Dpq = DistanceHalfPlane(p, q)
Append (VQ intersect Dpq) to Clips
VP = VPs[index of p in P]
EndClip = Union(Clips)
Append (VP subtract EndClip) to AgentControls

FactionControls = List()
For faction in F:
Agents = List of Agents in faction
Ctrl = List()
For agent in Agents:
Append AgentControls[index of agent in P] to Ctrl
Append Union(Ctrl) to FactionControls

Yield (AgentControls, FactionControls)

```

3. Proposed Approach

Looking at the straightforward baseline approach, we can make some optimization changes with regards to each individual process. One obvious optimization is in the computation of the visibility polygon, where in place of the $O(n^2)$ raycast method, the $O(n \log n)$ plane sweep algorithm of Asano [1] is used instead.

Another optimization is by aggressively pruning candidates for q . If $\Lambda_p \cap \Lambda_q$ or $\Lambda_p \cap D_{pq}$ yields $\emptyset$, then computing $\Lambda_q \cap D_{pq}$ is wholly unneeded, as the geometry will never intersect with Λ_p . We can optimize the $\Lambda_p \cap \Lambda_q$ predicate through spatial partitioning of Λ to ensure fast querying of intersecting regions, while culling with the $\Lambda_p \cap D_{pq}$ predicate by naïve edge checking to the distance line. The latter does consume $O(n)$ respective to the vertices of Λ_p but should be faster than the intersection arithmetic. These early-rejections reduce the number of times we compute $\Lambda_q \cap D_{pq}$, resulting in a minor speedup.

The total speedup should reduce the time complexity to be $O(S \log S)$ respective to geometry complexity, while being roughly $O(P^2 \log P)$ respective to agent count on average. The worst case respective to agent count remains quadratic.

4. Implementation Details

We implement the above pseudocode in a Jupyter Notebook using Python 3. Matplotlib is utilized for numerical manipulation and graphical display. Shapely is additionally used for easier manipulation of geometric data, and SciPy was used for its fast implementation of Voronoi construction in input generation. The math module makes an appearance for some standard functions, and the time module is used for benchmarking.

Parts of the code are AI-generated then verified for correctness with real inputs. We ensure that the baseline and proposed approaches follow the stated processes and we take full responsibility for the performance and correctness of the code.

5. Experimental Setup

The execution environment is a Python 3 Google Compute Engine backend with a Google Colab session on CPU hardware acceleration and the latest version Runtime (as of May 3, 2026.)

Inputs for the program are generated randomly by a factory process that generates inputs based on the

provided parameters, such as region size, agent count, obstacle complexity, and so on.

The following describes the test input batches and their specified parameters. Each input batch is run 20 times and the average runtime and variance of each process is measured.

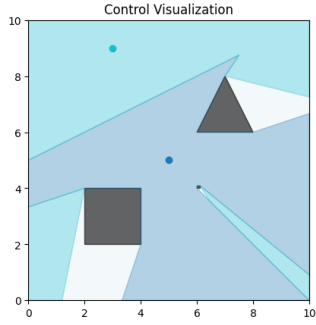
Table 2: Test Parameters

No.	Batch Detail	Notes
0	$\Omega_x, \Omega_y = 10$ $M = 0$ $F = 1$ $P = 1$	Measures the time taken for the trivial operation for both processes for reference.
1	$\Omega_x, \Omega_y = 10$ $M = 0$ $F = 2$ $P = 2$	Simple “open” scenario.
2	$\Omega_x, \Omega_y = 100$ $M = 10$ $3 \leq V(O) \leq 3$ $F = 2$ $P = 2$	Trivial obstacles. Introduce sparse, low-complexity geometry that barely affect region topology.
3	$\Omega_x, \Omega_y = 500$ $M = 10$ $3 \leq V(O) \leq 3$ $F = 4$ $P = 20$	Batch 2 but player count also increases. Since the runtime increase highly depends on P, slowing of processes is apparent.
4	$\Omega_x, \Omega_y = 100$ $M = 10$ $4 \leq V(O) \leq 6$ $F = 2$ $P = 2$	Introduce more complex structures. All other parameters remain the same as Batch 2 for a point of reference.
5	$\Omega_x, \Omega_y = 500$ $M = 10$ $6 \leq V(O) \leq 6$ $F = 4$ $P = 20$	Batch 3, but with hexagons everywhere.
6	$\Omega_x, \Omega_y = 1000$ $M = 100$ $3 \leq V(O) \leq 3$ $F = 2$ $P = 2$	Maximal region size and obstacle count. Other parameters are minimized for raw scaling.
7	$\Omega_x, \Omega_y = 1000$ $M = 0$ $F = 4$ $P = 40$	Maximal region size and player count. Obstacles are removed for raw scaling.
8	$\Omega_x, \Omega_y = 1000$ $M = 100$ $3 \leq V(O) \leq 3$ $F = 4$ $P = 40$	Maximized obstacles and players with simple geometry.

6. Results and Evaluation

For the correctness of the process, we have plotted the control regions of agents and factions represented on the playing region with the list of obstacles. Figure 2 describes one particular configuration with 2 agents and 4 obstacles.

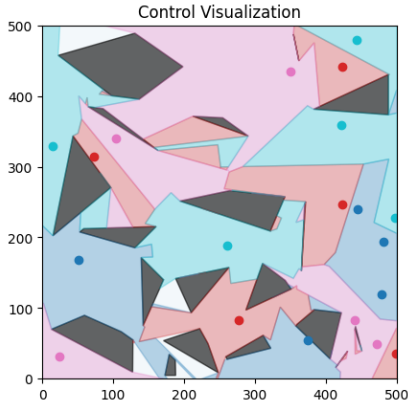
Figure 2. Control Region of 2 Agents



We see the characteristics of the control regions clearly in this figure. The agent in the center can see the subregion that is not occluded by the three obstacles (the square, triangle, and the small pebble) and as such controls that region, excluding where it loses in distance to the agent on the top. This agent, meanwhile, can see things that the center agent cannot, and as such have sole claim over those regions. Note that in spite of the center agent being closer to these “hidden regions”, it cannot put a claim on those regions as they are not visible.

As the number of agents, factions, and obstacles increase, we find the resultant graph to be even more chaotic. Figure 3 illustrates a configuration of, being placed randomly, 20 agents across 4 factions and 20 3- to 5-gon obstacles, where the regions are fragmented to an extent that you cannot make heads or tails of how a faction could possibly have controlled such a region.

Figure 3. A Chaotic 20-agent Control Game



As for the runtime of the processes, we find the results according to Table 3. Only batch 8 was executed in multiple sessions due to the Google Colab timeout.

Table 3: Process Runtime Results

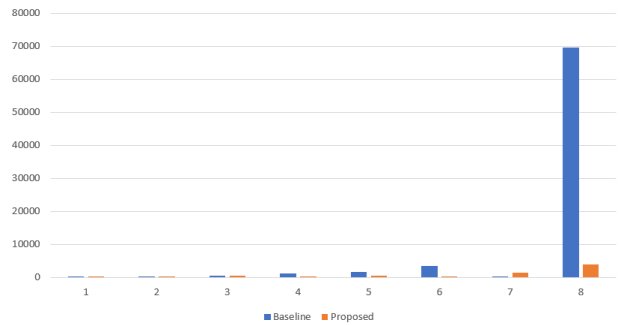
Batch	Baseline (s)		Proposed (s)	
	Mean	Std.Dev.	Mean	Std.Dev.
0	0.012608	0.002246	0.000951	0.000133
1	0.017046	0.002506	0.002784	0.000483
2	0.639550	0.143448	0.006661	0.001638
3	6.593035	0.805749	0.380003	0.052057
4	15.985022	1.664747	0.368785	0.078251
5	21.936642	0.806264	0.374763	0.054070
6	44.222147	1.697099	0.042865	0.009035
7	1.572709	0.303520	1.253981	0.196687
8	876.833466	36.494560	3.677297	0.814566

We hereby observe that, even in the trivial case (in batch 0) the runtime for the baseline process is many times that of the proposed process. This is due to the nature of Python implementations being dependent on the number of lines of code that require interpreting using Python’s slow runtime backend. We may assume, though to our disadvantage for reasons that will be apparent, that the work required to calculate the trivial scenario is equal, then we take the runtime of each process and normalize it as a ratio over the baseline. The resulting values are shown in Table 4 and then graphed onto a bar chart, though the difference of scale makes this chart not very useful for any particular use case.

Table 4: Normalized Runtime Results

Batch	Baseline (Ratio)		Proposed (Ratio)		Speedup
	Mean	Std.Dev.	Mean	Std.Dev.	
0	1.000x	0.178x	1.000x	0.140x	1.000x
1	1.352	0.199	2.927	0.508	0.462x
2	50.726	11.378	7.004	1.722	7.242x
3	522.925	63.908	399.583	54.739	1.309x
4	1267.848	132.039	387.787	82.283	3.269x
5	1739.899	63.949	394.073	56.856	4.415x
6	3507.467	134.605	45.074	9.501	77.82x
7	124.739	24.074	1318.592	206.821	0.095x
8	69545.802	2894.556	3866.769	856.536	17.99x

Figure 4. Bar Chart of Normalized Process Runtime



6.1. Discussion

Attentive readers should find the speedup not unusual in the least; after all, we only applied known improvements and proven algorithms to a naïve baseline process to make it faster, and faster it was: with very high speedup ratios in scenarios 6 and 8, it was made clear that the proposed approach could handle the scaling of higher geometric complexity much easier than the baseline, which confirms the estimated decrease in time complexity.

There is a blemish in the proposed algorithm, however, in that the runtime stalls in “open” scenarios — that is, where obstacles hardly impede visibility, and many agents compete over the same areas. In such cases, the early rejection employed by the proposed approach rarely functions, and the total process in the end takes longer due to the heavy cost of generating and querying temporary objects, such as the creation of the STRtree.

Considering that the time to calculate the “trivial” batch for both approaches are so far apart, however, it is also possible that the normalized values are overly lenient towards the baseline approach. After all, taking only raw runtime into account, the proposed approach runs at an approximate 10x faster than the baseline in basically every batch. The supposed speedup from implementation changes is difficult to analyze, and should require an implementation in another programming language for comparison.

In particularly simple geometries, the baseline approach may be considered for actual usage, though practical usages will likely see a healthy mix of obstacles and agents, in which case the improved approach has a respectable runtime — enough for real-life applications, and possibly even real-time rendering in programming languages not named Python.

6.2. Conclusion

We defined the visibility-constrained faction control problem and provided a naïve and later improved process to compute the solution. With the improvements, we could compute the control regions of factions at around 10 times the speed of the naïve approach, though the additional processing can add overhead in more trivial or open scenarios.

We can use this process to visualize the hidden factors behind gameplay decisions in some games. For example, in shooting games, the areas you cover with your bullet fire is very important. With a 3-dimensional variant of this problem, you would be solving the bullet cover of your team relative to the map, and you can adjust formation to cover a variety of scenarios. This will greatly improve strategizing in team-based shooting games, whether it be securing objectives or deathmatch

modes. It can also be used as a tool for spectators to follow along with the decisions of the players in tournament and high-stakes play.

7. References

- [1] T. Asano, “An efficient algorithm for finding the visibility polygon for a polygonal region with holes,” *IEICE Transactions*, vol. 68, no. 9, pp. 557–559, 1985.